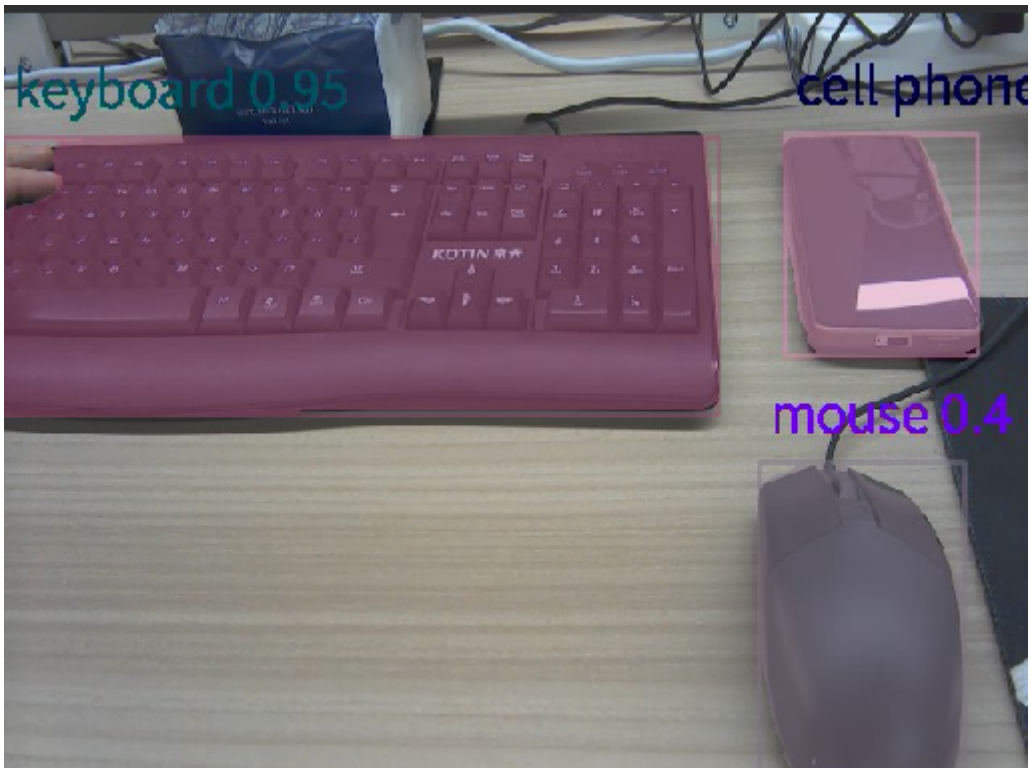
yolov8n 分割

yolov8n 分割
- 例程实验效果
- 代码讲解
  - 代码结构
  - 部分代码
  - 流程图

例程实验效果

本节我们来学习一下使用K230实现基于 yolov8n 的物体分割功能。

连接IDE后运行本节例程代码，用K230对准周围，可以看到在物体检测的基础上，被检测出的物体区域会用不同颜色渲染



代码讲解

代码结构

由于分割是基于检测而来的，所以这里我们主要列出和上一节检测的代码结构上的差异

1. 添加了分割模型特有的初始化阶段，包括掩码数组初始化
2. 修改了检测流程为检测分割流程，添加掩码生成步骤
3. 在显示流程中添加了分割掩码的绘制步骤
4. 调整了主循环中各个子流程的顺序和关系

部分代码

完整代码请参考文件【源码汇总 / 09.Scene / 04.segment_yolov8n.py】

```
    def postprocess(self, results):
        """
        自定义当前任务的后处理
        Custom post-processing for the current task

        参数:
        Parameters:
            results: 模型推理结果 / Model inference results

        返回:
        Returns:
            seg_res: 分割结果 / Segmentation results
        """
        with ScopedTiming("postprocess", self.debug_mode > 0):
            # 这里使用了aidemo的segment_postprocess接口进行后处理
            # Using aidemo's segment_postprocess interface for post-processing
            seg_res = aidemo.segment_postprocess(
                results,
                [self.rgb888p_size[1], self.rgb888p_size[0]],
                self.model_input_size,
                [self.display_size[1], self.display_size[0]],
                self.confidence_threshold,
                self.nms_threshold,
                self.mask_threshold,
                self.masks
            )
            return seg_res

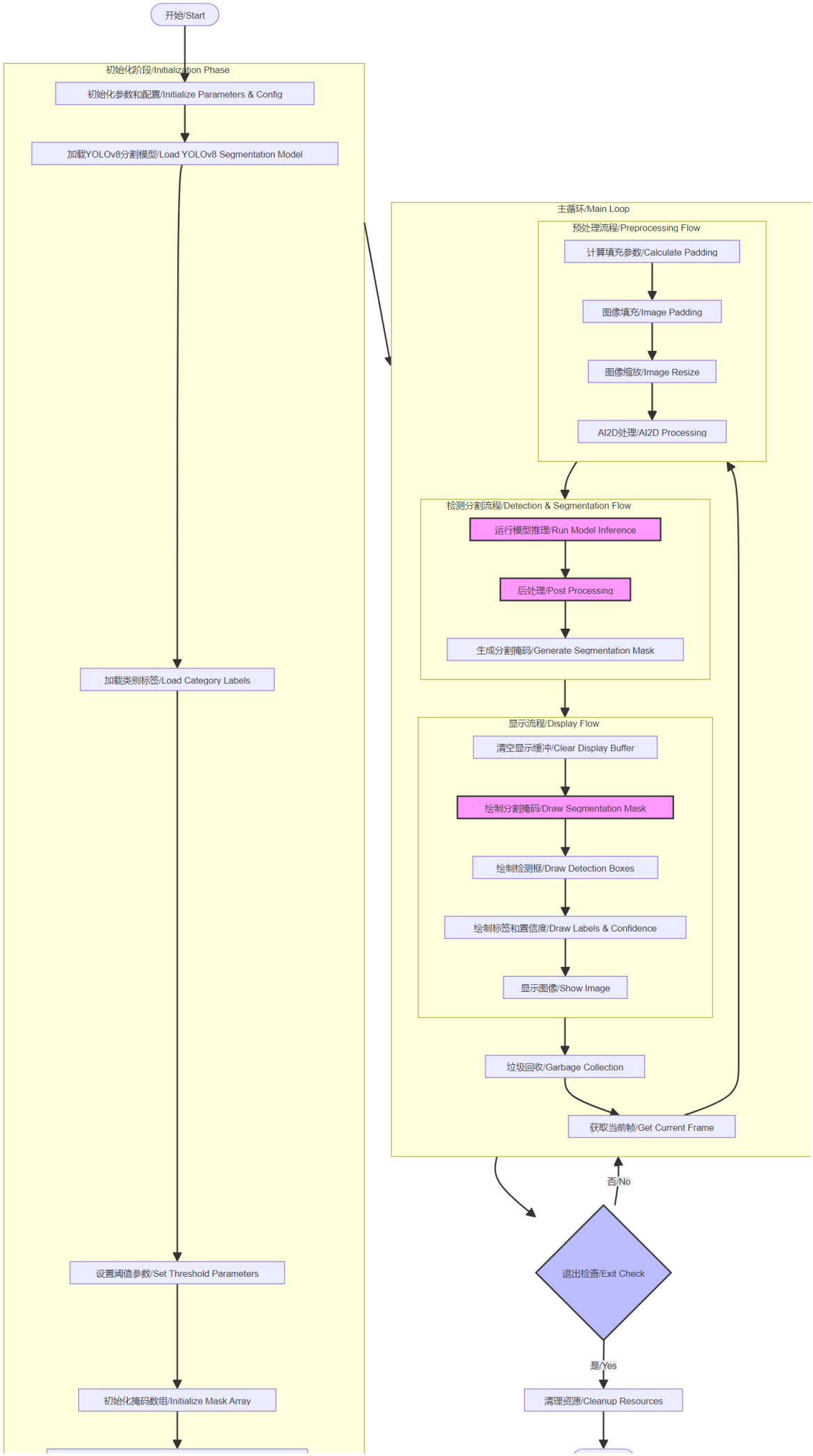
    def draw_result(self, pl, seg_res):
        """
        绘制分割结果到显示层
        Draw segmentation results to display layer

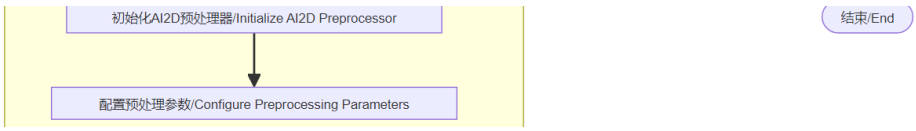
        参数:
        Parameters:
            pl: Pipeline对象 / Pipeline object
            seg_res: 分割结果 / Segmentation results
        """
        with ScopedTiming("display_draw", self.debug_mode > 0):
            if seg_res[0]:  # 如果有检测到物体 / If objects are detected
                pl.osd_img.clear()  # 清除OSD图层 / Clear OSD layer
                # 创建引用mask数据的图像对象 / Create image object referencing mask
data
                mask_img = image.Image(self.display_size[0],
self.display_size[1], image.ARGB8888, alloc=image.ALLOC_REF, data=self.masks)
                pl.osd_img.copy_from(mask_img)  # 复制mask图像到OSD层 / Copy mask
image to OSD layer

                # 提取检测结果 / Extract detection results
                dets, ids, scores = seg_res[0], seg_res[1], seg_res[2]
                for i, det in enumerate(dets):
                    # 绘制标签和置信度 / Draw label and confidence
                    x1, y1, w, h = map(lambda x: int(round(x, 0)), det)
```

```
                    pl.osd_img.draw_string_advanced(
                        x1, y1-50, 32,
                        " " + self.labels[int(ids[i])] + " " +
str(round(scores[i], 2)),
                        color=self.get_color(int(ids[i]))
                    )
            else:
                pl.osd_img.clear()  # 没有检测结果时清除OSD / Clear OSD when no
detection
```

流程图


开始/Start
初始化阶段/Initialization Phase
初始化参数和配置/Initialize Parameters & Config
加载YOLOv8分割模型/Load YOLOv8 Segmentation Model
加载类别标签/Load Category Labels
设置阈值参数/Set Threshold Parameters
初始化掩码数组/Initialize Mask Array
主循环/Main Loop
预处理流程/Preprocessing Flow
计算填充参数/Calculate Padding
图像填充/Image Padding
图像缩放/Image Resize
AI2D处理/AI2D Processing
检测分割流程/Detection & Segmentation Flow
运行模型推理/Run Model Inference
后处理/Post Processing
生成分割掩码/Generate Segmentation Mask
显示流程/Display Flow
清空显示缓冲/Clear Display Buffer
绘制分割掩码/Draw Segmentation Mask
绘制检测框/Draw Detection Boxes
绘制标签和置信度/Draw Labels & Confidence
显示图像/Show Image
垃圾回收/Garbage Collection
获取当前帧/Get Current Frame
退出检查/Exit Check
否/No
是/Yes
清理资源/Cleanup Resources


初始化AI2D预处理器/Initialize AI2D Preprocessor
配置预处理参数/Configure Preprocessing Parameters
结束/End